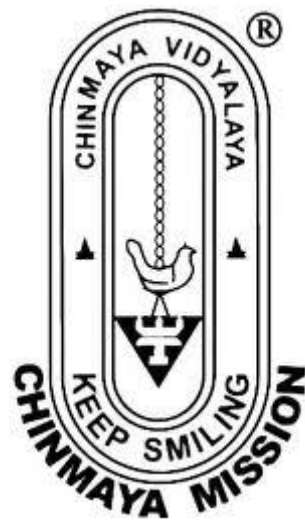


CHINMAYA VIDYALAYA
VIDYANAGAR
KASARAGOD



COMPUTER SCIENCE
INVESTIGATORY PROJECT

2022-2023

BIODATA

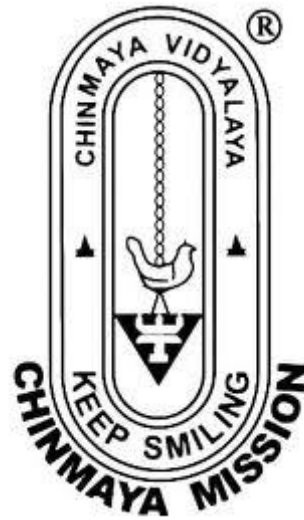
NAME: SRINIVAS

CLASS: XII

SUBJECT: COMPUTER SCIENCE

SCHOOL: CHINMAYA VIDYALAYA KASARAGOD

CERTIFICATE



THIS IS TO CERTIFY THAT A BONAFIDE STUDENT OF CLASS XII SRINIVAS OF CHINMAYA VIDYALAYA KASARAGOD HAS SUCESSFULLY WORKED AND COMPLETED ON PROJECT ENTITLED **“BANK MANAGEMENT SYSTEM IN PYTHON”** UNDER OUR SUPERVISION DURING THE **ACADEMIC YEAR 2022-23**

DATE:

PLACE: KASARAGOD

REGISTER NUMBER:

TEACHER IN CHARGE

SIGNATURE OF PRINCIPAL

[DEPT OF COMPUTER SCIENCE]

EXTERNAL EXAMINER

INTERNAL EXAMINER

ACKNOWLEDGMENT

IT IS WITH GREAT PLEASURE THAT I FIND MYSELF PENNING DOWN THESE LINES TO EXPRESS MY SINCERE THANKS TO ALL THOSE PEOPLE WHO HELPED ME A LONG WAY IN COMPLETE THIS PROJECT. THE HARMONIOUS CLIMATE IN OUR SCHOOL PROVIDED PROPER ATMOSPHERE FOR CREATING THIS PROJECT. IT WAS A PRIVILEGE TO HAVE BEEN GUIDED BY **MR. RAJESH SIR**. I WOULD ALSO LIKE TO EXTEND MY HEARTFELT GRATITUDE TO **PRINCIPAL SIR MR. BIJU MADATHIL** FOR THE SUPPORT TO COMPLETE THE PROJECT.

I AM SO GRATEFUL TO MY PARENTS AND CLASSMATES WHO HELPED ME DURING THE FINALIZATION OF MY PROJECT WITH THEIR CONSTRUCTIVE CRITICISM AND ADVICE.

INDEX

- ABOUT PYTHON
- ABOUT MySQL
- ABOUT BANK MANAGEMENT SYSTEM
- SOURCE CODE
- SCREENSHOT
- CONCLUSION AND FUTURE SCOPE
- BIBILOGRAPHY

ABOUT PYTHON

PYTHON IS AN INTERPRETED, OBJECT-ORIENTED, HIGH-LEVEL PROGRAMMING LANGUAGE WITH DYNAMIC SEMANTICS. ITS HIGH-LEVEL BUILT IN DATA STRUCTURES, COMBINED WITH DYNAMIC TYPING AND DYNAMIC BINDING, MAKE IT VERY ATTRACTIVE FOR RAPID APPLICATION DEVELOPMENT, AS WELL AS FOR USE AS A SCRIPTING OR GLUE LANGUAGE TO CONNECT EXISTING COMPONENTS TOGETHER. PYTHON'S SIMPLE, EASY TO LEARN SYNTAX EMPHASIZES READABILITY AND THEREFORE REDUCES THE COST OF PROGRAM MAINTENANCE. PYTHON SUPPORTS MODULES AND PACKAGES, WHICH ENCOURAGES PROGRAM MODULARITY AND CODE REUSE. THE PYTHON INTERPRETER AND THE EXTENSIVE STANDARD LIBRARY ARE AVAILABLE IN SOURCE OR BINARY FORM WITHOUT CHARGE FOR ALL MAJOR PLATFORMS, AND CAN BE FREELY DISTRIBUTED.

OFTEN, PROGRAMMERS FALL IN LOVE WITH PYTHON BECAUSE OF THE INCREASED PRODUCTIVITY IT

PROVIDES. SINCE THERE IS NO COMPILATION STEP, THE EDIT-TEST-DEBUG CYCLE IS INCREDIBLY FAST. DEBUGGING PYTHON PROGRAMS IS EASY: A BUG OR BAD INPUT WILL NEVER CAUSE A SEGMENTATION FAULT. INSTEAD, WHEN THE INTERPRETER DISCOVERS AN ERROR, IT RAISES AN EXCEPTION. WHEN THE PROGRAM DOESN'T CATCH THE EXCEPTION, THE INTERPRETER PRINTS A STACK TRACE. A SOURCE LEVEL DEBUGGER ALLOWS INSPECTION OF LOCAL AND GLOBAL VARIABLES, EVALUATION OF ARBITRARY EXPRESSIONS, SETTING BREAKPOINTS, STEPPING THROUGH THE CODE A LINE AT A TIME, AND SO ON. THE DEBUGGER IS WRITTEN IN PYTHON ITSELF, TESTIFYING TO PYTHON'S INTROSPECTIVE POWER. ON THE OTHER HAND, OFTEN THE QUICKEST WAY TO DEBUG A PROGRAM IS TO ADD A FEW PRINT STATEMENTS TO THE SOURCE: THE FAST EDIT-TEST-DEBUG CYCLE MAKES THIS SIMPLE APPROACH VERY EFFECTIVE.

ABOUT MySQL

MySQL IS A FAST, EASY-TO-USE RDBMS BEING USED FOR MANY SMALL AND BIG BUSINESSES. MySQL IS DEVELOPED, MARKETING AND SUPPORTED BY MYSQL AB, WHICH IS A SWEDISH COMPANY. MySQL IS BECOMING SO POPULAR BECAUSE OF MANY GOOD REASONS -

- MySQL IS RELEASED UNDER AN OPEN-SOURCE LICENSE. SO, YOU HAVE NOTHING TO PAY TO USE IT.
- MySQL IS A VERY POWERFUL PROGRAM IN ITS OWN RIGHT. IT HANDLES A LARGE SUBSET OF THE FUNCTIONALITY OF THE MOST EXPENSIVE AND POWERFUL DATABASE PACKAGES.
- MySQL USES A STANDARD FORM OF THE WELL-KNOWN SQL DATA LANGUAGE.
- MySQL WORKS ON MANY OPERATING SYSTEMS AND WITH MANY LANGUAGES INCLUDING PHP, PERL, C, C++, JAVA, ETC.
- MySQL WORKS VERY QUICKLY AND WORKS WELL EVEN WITH LARGE DATA SETS.

- MySQL IS VERY FRIENDLY TO PHP, THE MOST APPRECIATED LANGUAGE FOR WEB DEVELOPMENT.
- MySQL SUPPORTS LARGE DATABASES, UP TO 50 MILLION ROWS OR MORE IN A TABLE. THE DEFAULT FILE SIZE LIMIT FOR A TABLE IS 4GB, BUT YOU CAN INCREASE THIS (IF YOUR OPERATING SYSTEM CAN HANDLE IT) TO A THEORETICAL LIMIT OF 8 MILLION TERABYTES (TB).
- MySQL IS CUSTOMIZABLE. THE OPEN-SOURCE GPL LICENSE ALLOWS PROGRAMMERS TO MODIFY THE MySQL SOFTWARE TO FIT THEIR OWN SPECIFIC ENVIRONMENTS.

ABOUT BANK MANAGEMENT SYSTEM

- BANK MANAGEMENT SYSTEM IS BASED ON PYTHON
- IT IS USED TO KEEP THE RECORDS OF CLIENTS, EMPLOYEE ETC IN BANK.
- THE BANK MANAGEMENT SYSTEM IS AN APPLICATION FOR MAINTAINING A PERSON, HAVING CURRENT[C], SAVINGS[S] ACCOUNT IN A BANK.
- THE SYSTEM PROVIDES THE ACCESS TO THE CUSTOMER TO CREATE AN ACCOUNT, DEPOSIT/WITHDRAW THE CASH FROM HIS ACCOUNT, ALSO TO VIEW REPORTS OF ALL ACCOUNTS PRESENT. THE FOLLOWING PRESENTATION PROVIDES THE SPECIFICATION FOR THE SYSTEM.

SOURCE CODE

```
import pickle
```

```
import os
```

```
import pathlib
```

```
class Account :
```

```
    accNo = 0
```

```
    name = "
```

```
    deposit=0
```

```
    type = "
```

```
    def createAccount(self):
```

```
        self.accNo= int(input("Enter The Account No : "))
```

```
        self.name = input("Enter The Account Holder Name : ")
```

```
        self.type = input("Ente The Type Of Account [C/S] : ")
```

```
self.deposit = int(input("Enter The Initial Amount(>=500 for Saving  
and >=1000 for current):"))
```

```
print("\n\n\nAccount Created")
```

```
def showAccount(self):
```

```
    print("Account Number : ",self.accNo)
```

```
    print("Account Holder Name : ", self.name)
```

```
    print("Type of Account",self.type)
```

```
    print("Balance : ",self.deposit)
```

```
def modifyAccount(self):
```

```
    print("Account Number : ",self.accNo)
```

```
    self.name = input("Modify Account Holder Name :")
```

```
    self.type = input("Modify Type of Account :")
```

```
    self.deposit = int(input("Modify Balance :"))
```

```
def depositAmount(self,amount):
```

```
    self.deposit += amount
```

```
def withdrawAmount(self,amount):
```

```
    self.deposit -= amount
```

```
def report(self):
```

```
    print(self.accNo, " ",self.name ," ",self.type," ", self.deposit)
```

```
def getAccountNo(self):
```

```
    return self.accNo
```

```
def getAcccountHolderName(self):
```

```
    return self.name
```

```
def getAccountType(self):
```

```
    return self.type
```

```
def getDeposit(self):
```

```
        return self.deposit
```

```
def intro():
```

```
    print("\t\t\t*****")
```

```
    print("\t\t\tBANK MANAGEMENT SYSTEM")
```

```
    print("\t\t\t*****")
```

```
    input()
```

```
def writeAccount():
```

```
    account = Account()
```

```
    account.createAccount()
```

```
    writeAccountsFile(account)
```

```
def displayAll():
```

```
    file = pathlib.Path("accounts.data")
```

```
    if file.exists ():
```

```
infile = open('accounts.data','rb')
```

```
mylist = pickle.load(infile)
```

```
for item in mylist :
```

```
    print(item.accNo," ", item.name, " ",item.type, " ",item.deposit )
```

```
infile.close()
```

```
else :
```

```
    print("No records to display")
```

```
def displaySp(num):
```

```
    file = pathlib.Path("accounts.data")
```

```
    if file.exists ():
```

```
        infile = open('accounts.data','rb')
```

```
        mylist = pickle.load(infile)
```

```
        infile.close()
```

```
found = False
```

```
for item in mylist :
```

```
    if item.accNo == num :
```

```
        print("Your Account Balance Is = ",item.deposit)
```

```
        found = True
```

```
    else :
```

```
        print("No Records To Search")
```

```
if not found :
```

```
    print("No Existing Record With This Number")
```

```
def depositAndWithdraw(num1,num2):
```

```
    file = pathlib.Path("accounts.data")
```

```
    if file.exists ():
```



```
infile = open('accounts.data','rb')
```

```
mylist = pickle.load(infile)
```

```
infile.close()
```

```
os.remove('accounts.data')
```

```
for item in mylist :
```

```
    if item.accNo == num1 :
```

```
        if num2 == 1 :
```

```
            amount = int(input("Enter The Amount To Deposit : "))
```

```
            item.deposit += amount
```

```
            print("Your Account Is Updated")
```

```
        elif num2 == 2 :
```

```
            amount = int(input("Enter the amount to withdraw : "))
```

```
            if amount <= item.deposit :
```

```
item.deposit -=amount
```

```
else :
```

```
print("You Cannot Withdraw Larger Amount")
```

```
else :
```

```
print("No Records To Search")
```

```
outfile = open('newaccounts.data','wb')
```

```
pickle.dump(mylist, outfile)
```

```
outfile.close()
```

```
os.rename('newaccounts.data', 'accounts.data')
```

```
def deleteAccount(num):
```

```
file = pathlib.Path("accounts.data")
```

```
if file.exists ():
```

```
infile = open('accounts.data','rb')
```

```
oldlist = pickle.load(infile)
```

```
infile.close()
```

```
newlist = []
```

```
for item in oldlist :
```

```
    if item.accNo != num :
```

```
        newlist.append(item)
```

```
os.remove('accounts.data')
```

```
outfile = open('newaccounts.data','wb')
```

```
pickle.dump(newlist, outfile)
```

```
outfile.close()
```

```
os.rename('newaccounts.data', 'accounts.data')
```

```
def modifyAccount(num):
```

```
    file = pathlib.Path("accounts.data")
```

```
if file.exists ():
```

```
    infile = open('accounts.data','rb')
```

```
    oldlist = pickle.load(infile)
```

```
    infile.close()
```

```
    os.remove('accounts.data')
```

```
    for item in oldlist :
```

```
        if item.accNo == num :
```

```
            item.name = input("Enter The Account Holder Name : ")
```

```
            item.type = input("Enter The Account Type : ")
```

```
            item.deposit = int(input("Enter The Amount : "))
```

```
    outfile = open('newaccounts.data','wb')
```

```
    pickle.dump(oldlist, outfile)
```

```
    outfile.close()
```

```
os.rename('newaccounts.data', 'accounts.data')
```

```
def writeAccountsFile(account) :
```

```
    file = pathlib.Path("accounts.data")
```

```
    if file.exists ():
```

```
        infile = open('accounts.data','rb')
```

```
        oldlist = pickle.load(infile)
```

```
        oldlist.append(account)
```

```
        infile.close()
```

```
        os.remove('accounts.data')
```

```
    else :
```

```
        oldlist = [account]
```

```
    outfile = open('newaccounts.data','wb')
```

```
    pickle.dump(oldlist, outfile)
```

```
outfile.close()
```

```
os.rename('newaccounts.data', 'accounts.data')
```

```
# start of the program
```

```
ch=""
```

```
num=0
```

```
intro()
```

```
while ch != 8:
```

```
    #system("cls");
```

```
    print("\tMAIN MENU")
```

```
    print("\t1. NEW ACCOUNT")
```

```
    print("\t2. DEPOSIT AMOUNT")
```

```
    print("\t3. WITHDRAW AMOUNT")
```

```
print("\t4. BALANCE ENQUIRY")
```

```
print("\t5. ALL ACCOUNT HOLDER LIST")
```

```
print("\t6. CLOSE AN ACCOUNT")
```

```
print("\t7. MODIFY AN ACCOUNT")
```

```
print("\t8. EXIT")
```

```
print("\tSelect Your Option (1-8) ")
```

```
ch = input()
```

```
#system("cls");
```

```
if ch == '1':
```

```
    writeAccount()
```

```
elif ch == '2':
```

```
    num = int(input("\tEnter The Account No. : "))
```

```
depositAndWithdraw(num, 1)
```

```
elif ch == '3':
```

```
num = int(input("\tEnter The Account No. : "))
```

```
depositAndWithdraw(num, 2)
```

```
elif ch == '4':
```

```
num = int(input("\tEnter The Account No. : "))
```

```
displaySp(num)
```

```
elif ch == '5':
```

```
displayAll();
```

```
elif ch == '6':
```

```
num =int(input("\tEnter The Account No. : "))
```

```
deleteAccount(num)
```

```
elif ch == '7':
```



```
num = int(input("\tEnter The Account No. : "))
```

```
modifyAccount(num)
```

```
elif ch == '8':
```

```
print("\tThank You For Using Bank Management System")
```

```
break
```

```
else :
```

```
print("Invalid Choice")
```

```
ch = input("Enter Your Choice : ")
```

SCREENSHOT

```
*****  
BANK MANAGEMENT SYSTEM  
*****
```

```
MAIN MENU  
1. NEW ACCOUNT  
2. DEPOSIT AMOUNT  
3. WITHDRAW AMOUNT  
4. BALANCE ENQUIRY  
5. ALL ACCOUNT HOLDER LIST  
6. CLOSE AN ACCOUNT  
7. MODIFY AN ACCOUNT  
8. EXIT  
Select Your Option (1-8)
```

BANK MANAGEMENT SYSTEM

MAIN MENU
1. NEW ACCOUNT
2. DEPOSIT AMOUNT
3. WITHDRAW AMOUNT
4. BALANCE ENQUIRY
5. ALL ACCOUNT HOLDER LIST
6. CLOSE AN ACCOUNT
7. MODIFY AN ACCOUNT
8. EXIT
Select Your Option (1-8)

1
Enter The Account No : 1001
Enter The Account Holder Name : SRINIVAS
Enter The Type Of Account [C/S] : S
Enter The Initial Amount(>=500 for Saving and >=1000 for current):1000

Account Created
Enter Your Choice :
MAIN MENU
1. NEW ACCOUNT
2. DEPOSIT AMOUNT
3. WITHDRAW AMOUNT
4. BALANCE ENQUIRY
5. ALL ACCOUNT HOLDER LIST
6. CLOSE AN ACCOUNT
7. MODIFY AN ACCOUNT
8. EXIT
Select Your Option (1-8)

1
Enter The Account No : 1002
Enter The Account Holder Name : SRINIVASA
Enter The Type Of Account [C/S] : C
Enter The Initial Amount(>=500 for Saving and >=1000 for current):5000

Account Created

BANK MANAGEMENT SYSTEM

MAIN MENU

1. NEW ACCOUNT
2. DEPOSIT AMOUNT
3. WITHDRAW AMOUNT
4. BALANCE ENQUIRY
5. ALL ACCOUNT HOLDER LIST
6. CLOSE AN ACCOUNT
7. MODIFY AN ACCOUNT
8. EXIT

Select Your Option (1-8)

2

Enter The Account No. : 1001

Enter The Amount To Deposit : 2000

Your Account Is Updated

Enter Your Choice :

MAIN MENU

1. NEW ACCOUNT
2. DEPOSIT AMOUNT
3. WITHDRAW AMOUNT
4. BALANCE ENQUIRY
5. ALL ACCOUNT HOLDER LIST
6. CLOSE AN ACCOUNT
7. MODIFY AN ACCOUNT
8. EXIT

Select Your Option (1-8)

2

Enter The Account No. : 1002

Enter The Amount To Deposit : 2000

Your Account Is Updated

BANK MANAGEMENT SYSTEM

MAIN MENU
1. NEW ACCOUNT
2. DEPOSIT AMOUNT
3. WITHDRAW AMOUNT
4. BALANCE ENQUIRY
5. ALL ACCOUNT HOLDER LIST
6. CLOSE AN ACCOUNT
7. MODIFY AN ACCOUNT
8. EXIT
Select Your Option (1-8)

3

Enter The Account No. : 1001
Enter the amount to withdraw : 1000
Enter Your Choice :

MAIN MENU
1. NEW ACCOUNT
2. DEPOSIT AMOUNT
3. WITHDRAW AMOUNT
4. BALANCE ENQUIRY
5. ALL ACCOUNT HOLDER LIST
6. CLOSE AN ACCOUNT
7. MODIFY AN ACCOUNT
8. EXIT
Select Your Option (1-8)

3

Enter The Account No. : 1002
Enter the amount to withdraw : 1000
Enter Your Choice :

BANK MANAGEMENT SYSTEM

MAIN MENU
1. NEW ACCOUNT
2. DEPOSIT AMOUNT
3. WITHDRAW AMOUNT
4. BALANCE ENQUIRY
5. ALL ACCOUNT HOLDER LIST
6. CLOSE AN ACCOUNT
7. MODIFY AN ACCOUNT
8. EXIT
Select Your Option (1-8)

4

Enter The Account No. : 1001
Your Account Balance Is = 2000
Enter Your Choice :

MAIN MENU
1. NEW ACCOUNT
2. DEPOSIT AMOUNT
3. WITHDRAW AMOUNT
4. BALANCE ENQUIRY
5. ALL ACCOUNT HOLDER LIST
6. CLOSE AN ACCOUNT
7. MODIFY AN ACCOUNT
8. EXIT
Select Your Option (1-8)

4

Enter The Account No. : 1002
Your Account Balance Is = 6000
Enter Your Choice : |

BANK MANAGEMENT SYSTEM

MAIN MENU

1. NEW ACCOUNT
2. DEPOSIT AMOUNT
3. WITHDRAW AMOUNT
4. BALANCE ENQUIRY
5. ALL ACCOUNT HOLDER LIST
6. CLOSE AN ACCOUNT
7. MODIFY AN ACCOUNT
8. EXIT

Select Your Option (1-8)

5

1001	SRINIVAS	S	2000
1002	SRINIVASA	C	6000

Enter Your Choice :

BANK MANAGEMENT SYSTEM

MAIN MENU
1. NEW ACCOUNT
2. DEPOSIT AMOUNT
3. WITHDRAW AMOUNT
4. BALANCE ENQUIRY
5. ALL ACCOUNT HOLDER LIST
6. CLOSE AN ACCOUNT
7. MODIFY AN ACCOUNT
8. EXIT
Select Your Option (1-8)

6

Enter The Account No. : 1002

Enter Your Choice :

MAIN MENU
1. NEW ACCOUNT
2. DEPOSIT AMOUNT
3. WITHDRAW AMOUNT
4. BALANCE ENQUIRY
5. ALL ACCOUNT HOLDER LIST
6. CLOSE AN ACCOUNT
7. MODIFY AN ACCOUNT
8. EXIT
Select Your Option (1-8)

5

1001 SRINIVAS S 2000

Enter Your Choice : |

BANK MANAGEMENT SYSTEM

MAIN MENU

1. NEW ACCOUNT
2. DEPOSIT AMOUNT
3. WITHDRAW AMOUNT
4. BALANCE ENQUIRY
5. ALL ACCOUNT HOLDER LIST
6. CLOSE AN ACCOUNT
7. MODIFY AN ACCOUNT
8. EXIT

Select Your Option (1-8)

7

Enter The Account No. : 1001

Enter The Account Holder Name : SRINIVAS

Enter The Account Type : C

Enter The Amount : 3000

Enter Your Choice : |

BANK MANAGEMENT SYSTEM

MAIN MENU
1. NEW ACCOUNT
2. DEPOSIT AMOUNT
3. WITHDRAW AMOUNT
4. BALANCE ENQUIRY
5. ALL ACCOUNT HOLDER LIST
6. CLOSE AN ACCOUNT
7. MODIFY AN ACCOUNT
8. EXIT
Select Your Option (1-8)

Thank You For Using Bank Managemnt System

8

>>> |

CONCLUSION & FUTURE SCOPE

THIS PROJECT IS DEVELOPED TO NURTURE THE NEEDS OF A USER IN A BANKING SECTOR BY EMBEDDING ALL THE TASKS OF TRANSACTION TAKING PLACE IN BANK

FUTURE SCOPE PYTHON SUPPORTS BOTH FUNCTION-ORIENTED & STRUCTURE-ORIENTED PROGRAMMING. IT HAS FEATURES OF DYNAMIC MEMORY MANAGEMENT WHICH CAN MAKE USE OF COMPUTATIONAL RESOURCES EFFICIENTLY. IT IS ALSO COMPATIBLE WITH ALL POPULAR OPERATING SYSTEMS AND PLATFORMS. HENCE THIS LANGUAGE CAN BE UNIVERSALLY ACCEPTED BY ALL PROGRAMMERS.

FUTURE DIGITAL BANKING LIKE UPI, NETBANKING WILL ENHANCE BANKING SECTOR TO MAKE CONTACTLESS TRANSACTION ONLINE.

BY UPGRADING THE PYTHON CODE, DATA OF CUSTOMER CAN BE STORED IN SQL SERVER WHICH WILL MAKE IT CAPABLE TO DO INTERBANK TRANSACTION.

BIBLIOGRAPHY

WEBSITE:

PROJECTWORLDS.IN

PYTHON.ORG

BOOK:

COMPUTER SCIENCE WITH PYTHON PREETI ARORA